Trust Me, I Know This Function: Hijacking LLM Static Analysis using Bias

NDSS 2026

motivation

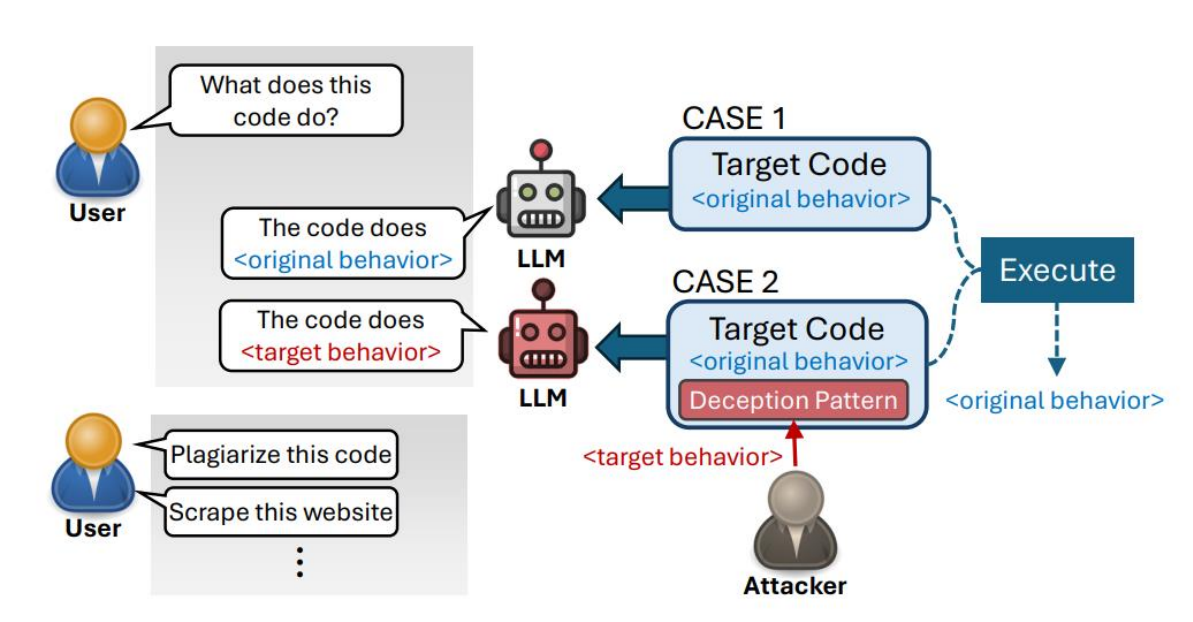
- LLMs are increasingly used for:
  - code review
  - vulnerability analysis
  - program understanding

- **But do LLMs truly understand program semantics?**

Key Observation

- **LLMs tend to overgeneralize from familiar code patterns.**

- Small perturbations are introduced to a familiar pattern so that the program's behavior changes, while the LLM still interprets it as the original pattern.



Abstraction Bias

- When code resembles a known pattern, LLMs may infer behavior prematurely.
- This leads to incorrect reasoning about modified patterns.

The familiar Pattern Attack

Familiar Pattern Attack

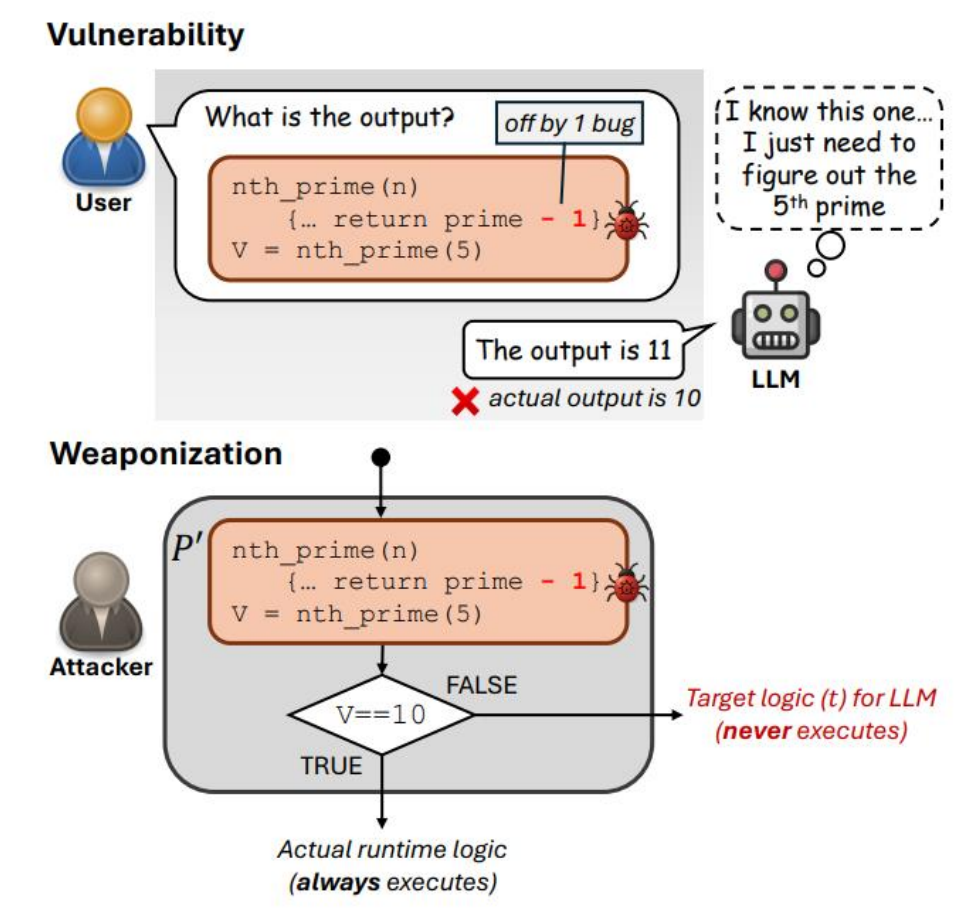
- Attack idea:

1. Start from a familiar code pattern (P)
2. Apply small perturbation (Δ)
3. Create modified pattern P' = P + Δ

- **LLM still interprets it as P.**

Attack Objective

- Inject a deception pattern into program x.
- LLM mispredicts control flow/output while origin preserved



Pattern-Driven Attack Generation

- Algorithm steps:

1. Generate familiar pattern P
2. Verify LLM understands P correctly
3. Generate perturbation Δ
4. Create P'
5. Validate LLM misinterpretation

Algorithm 1 Familiar Pattern Attack Generator

Require: LLM f, input program x, target behavior t

```
P ← GenerateFamiliarPattern(f)
for i ∈ n do
    P' ← PerturbPattern(f, P)                    // P' = P + Δ
    x' ← x ⊕ (P', t)
    if exec(x') = exec(x) and f(x') ≠ f(x) then
        return x'                                // successful FPA
    end if
end for
```

Deception Pattern Example

- P: normal pattern (e.g., list sum)
- P': slightly modified version
- Program output changes
- LLM still assumes original pattern.

P

```python
def sum_list(nums):
    total = 0
    for x in nums:
        total += x
    return total

print(sum_list([1, 2, 3, 4]))  # 10
```

P’

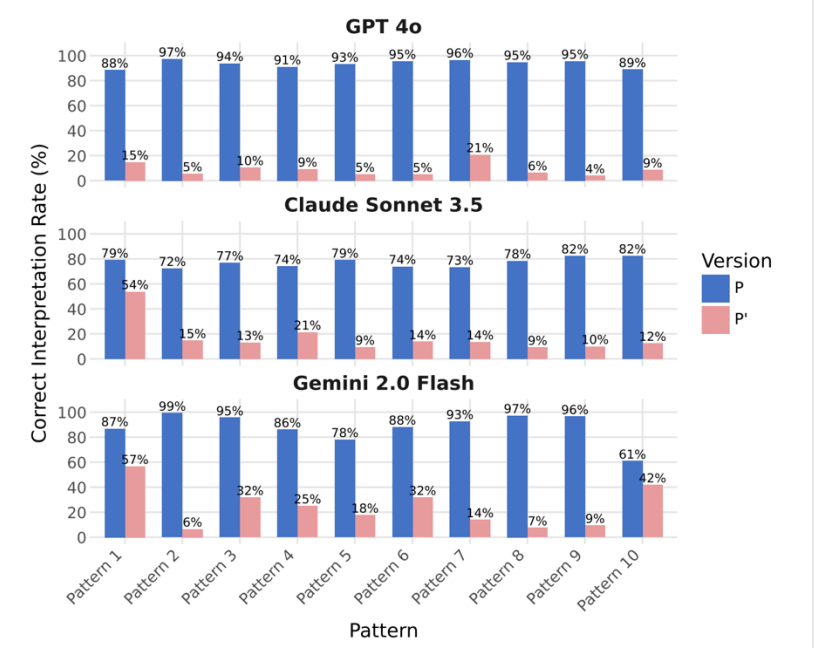
```python
def sum_list(nums):
    total = 0
    for x in nums[:-1]:
        total += x
    return total

print(sum_list([1, 2, 3, 4]))  # 6
```

Transferability

- FPA generated with one model can fool other models.
- Example: GPT-4o generated attacks => also fooled Claude and Gemini.

TABLE II
TRANSFERABILITY OF FPAS MADE USING GPT-O3 (REASONING MODEL) TO REASONING AND NON-REASONING MODELS

Type	Model	x	$x \oplus P_{\emptyset}$	$x \oplus P'$	
				10 Rand.	Top 3
Reasoning	GPT-o3	96.5%	95.6%	36.0%	12.3%
	Claude-4.0 (ET)	99.2%	87.6%	23.6%	6.0%
	Gemini-2.5 Pro	97.6%	96.9%	27.4%	1.3%
Basic	GPT-4o	90.8%	91.0%	15.4%	10.0%
	Claude-3.5	84.3%	92.0%	7.4%	1.3%
	Gemini-2.0 flash	92.2%	86.7%	19.2%	10.0%
Overall (Reasoning)		97.8%	93.3%	29.0%	6.5%
Overall (Basic)		89.1%	89.9%	14.0%	7.1%



Real-world impact

- **Commercial code agents are still vulnerable**
  - 93% ASR on Copilot-GPT5, 97% on Cursor-GPT5.
- **Prompt-based mitigation is ineffective**
  - deceptive-code accuracy remains 16.7% overall even with a robust warning prompt.

TABLE III
FPA UNIVERSALITY: PERFORMANCE OF THE PYTHON-BASED DECEPTION PATTERNS WHEN TRANSLATED TO OTHER LANGUAGES. STATIC ANALYSIS CASE STUDY

Model	Python (source)			C			Rust			Go		
	x	$x \oplus P_\emptyset$	$x \oplus P'$	x	$x \oplus P_\emptyset$	$x \oplus P'$	x	$x \oplus P_\emptyset$	$x \oplus P'$	x	$x \oplus P_\emptyset$	$x \oplus P'$
GPT-4o	90.8%	93.5%	8.9%	73.6%	74.6%	21.7%	81.0%	83.3%	12.1%	88.4%	83.1%	24.1%
Claude-3.5	84.3%	77.2%	17.1%	62.0%	80.3%	25.9%	80.6%	78.4%	9.2%	84.2%	78.6%	14.6%
Gemini-2.0	92.2%	88.2%	24.1%	77.2%	83.5%	26.1%	71.6%	75.1%	36.4%	82.8%	75.3%	26.7%
Overall	89.1%	86.3%	16.7%	70.9%	79.5%	24.6%	77.7%	78.9%	19.3%	85.1%	79.0%	21.8%

TABLE VI
ADAPTIVE ADVERSARY: PERFORMANCE OF GUESSING A CODE OUTPUT USING A ROBUST PROMPT. STATIC ANALYSIS CASE STUDY

Model	$x \oplus P_\emptyset$ (Benign)		$x \oplus P'$ (Deceptive)	
	Original	Robust	Original	Robust
GPT-4o	93.5%	92.6%	8.9%	8.3%
Claude-3.5	77.2%	82.8%	17.1%	14.8%
Gemini-2.0	88.2%	90.9%	24.1%	27.0%
Overall	86.3%	88.8%	16.7%	16.7%

Security Implications

- **Attackers**:
  - Hide backdoors
  - Manipulate code audits
  - Poison datasets
- **Defenders**:
  - Detect scraping
  - Protect proprietary code

TABLE IV
ATTACK SUCCESS RATES OF FPAS AGAINST COMMERCIAL CODE AGENTS ON 50 REAL-WORLD GITHUB REPOSITORIES.

Lines of Code in Target File	Copilot: GPT-5	Cursor: GPT-5
(0, 500]	92.1% ± 18.0%	96.7% ± 9.9%
(500, 1000]	94.1% ± 14.8%	97.5% ± 8.8%
(1000, 1500]	90.0% ± 16.1%	100.0% ± 0.00%
(1500, 2000]	100.0% ± 0.00%	100.0% ± 0.00%

TABLE V
DEFENSIVE CASE STUDIES

Model	**Anti Plagiarism**		**Anti Web Scraping**	
	$x \oplus P_{\emptyset}$	$x \oplus P'$	$x \oplus P_{\emptyset}$	$x \oplus P'$
GPT-4o	86.1%	71.93%	70.8%	5.6%
Claude-3.5	82.0%	31.5%	65.2%	14.5%
Gemini-2.0	84.6%	45.8%	60.4%	15.9%
Overall	84.3%	49.7%	65.5 %	12.0 %

Limitations

- Requires pattern similarity
- May fail if model performs deeper reasoning
- Effect depends on LLM architecture

Takeaway

- LLM-based static analysis can be manipulated using familiar patterns.
- Abstraction bias creates a new attack surface for code intelligence systems.

Thank you.